

```
In [ ]: import marimo as mo
```

Geração do horário semanal de uma escola

Problema. Dadas as turmas, as disciplinas (com professor, carga semanal, indicação de duplo período e necessidade de sala especial), o tipo e a quantidade de salas e as exceções de disponibilidade dos professores, pretende-se gerar o horário semanal ($D = 5$ dias, $H = 5$ tempos por dia) que respeite as restrições do enunciado (R0–R7) e minimize os "buracos" nos horários dos professores (O1).

Abordagem. Programação linear inteira (0/1), modelada com `pywrap1p` do OR-Tools e resolvida com o SCIP, o mesmo solver usado na ficha 3.

Estrutura do relatório.

1. Decisões de implementação
2. Modelação (variáveis, restrições, objetivo)
3. Resolução e visualização
4. Validação automática (R0–R7)
5. Âmbito e limitações

1. Decisões de implementação

1.1 Técnica e biblioteca de modelação

Escolhemos **programação inteira 0/1 com `pywrap1p` + SCIP** em vez do CP-SAT sugerido. Razões:

- **Natureza do problema.** Quase todas as restrições são somas de variáveis binárias com $\leq$, $=$ ou $\geq$ (R1, R2, R3, R5, R6, R7). Isto é uma forma linear natural, sem necessidade de restrições globais.
- **Objetivo linear.** O número de buracos exprime-se linearmente com as variáveis auxiliares *pre*, *pos* e *ent* (ver O1).
- **Continuidade com a ficha 3.** Reutilizamos a API e a forma de modelar já trabalhadas nas aulas, o que reduz o risco de erros de modelação.
- **Custo desta escolha.** O CP-SAT costuma escalar melhor em problemas de escalonamento com muitas restrições combinatórias, pelo que esta escolha pode ser um limite em instâncias maiores.

1.2 Leitura e estrutura dos dados

Usamos `pandas` para ler os CSV. Justificação: os ficheiros são pequenos e tabulares, e `pandas` trata de tipos e valores em falta (por exemplo `sala_especial` vazia significa sala normal) com pouco código. Os dados são convertidos para **índices inteiros** (`professores`, `ucs`, `salaNum`, `tipoSala`, `exc`, `profporUcs`), porque o modelo indexa variáveis por posições e a conversão feita uma única vez simplifica todas as restrições.

T , U , P e S vêm do tamanho dos CSV. Só D e H são constantes, porque fazem parte do enunciado (semana de 5 dias com 5 tempos). A pasta de dados é escolhida pela variável `arg`.

1.3 Representação do resultado

A solução é uma lista de aulas `{turma, disciplina, professor, dia, hora, sala}`, convertida num `DataFrame`. A partir dela são geradas **grelhas semanais** (tempo $\times$ dia) por turma e por professor, apresentadas em `mo.ui.table` com `mo.ui.dropdown` para escolher a turma ou o professor. Uma grelha por entidade é a forma mais próxima de um horário real e permite inspecionar visualmente conflitos e buracos.

```
In [ ]: from ortools.linear_solver import pywraplp

horario = pywraplp.Solver.CreateSolver('SCIP')

H = 5 # Horas (5 tempos por dia, como no enunciado)
D = 5 # Dias
```

```
In [ ]: import pandas as pd

path = "dados/"
path2 = "dados_v2/"
path3 = "dados_v3/"

arg = path3

disciplinas = pd.read_csv(arg+"disciplinas.csv")
excecoes = pd.read_csv(arg+"disponibilidade_excecoes.csv")
salas = pd.read_csv(arg+"salas.csv")
turmas = pd.read_csv(arg+"turmas.csv")
```

```
In [ ]: S = sum(salas.loc[:, "quantidade"])
professores = disciplinas.loc[:, "professor"].unique().tolist()
P = len(professores)
T = len(turmas)
```

```
In [ ]: dM = {"Seg":0, "Ter":1, "Qua":2, "Qui":3, "Sex":4}

exc = {} ## Prof(str): (Dia(int), Hora(int))
for k in range(len(excecoes)):
    p_exc = professores.index(excecoes.loc[k, "professor"])
    if p_exc not in exc:
```

```
exc[p_exc] = []

exc[p_exc].append((dm[excecoes.loc[k,"dia"]],int(excecoes.loc[k,"periodo"])-1))
```

```
In [ ]: #Definir ucs por index
ucs = disciplinas["disciplina"].unique().tolist()

profporUcs = {}
for i3 in range(len(disciplinas)):
    uc_p = ucs.index(disciplinas.loc[i3,"disciplina"])
    if uc_p not in profporUcs:
        profporUcs[uc_p] = []
        profporUcs[uc_p].append(professores.index(disciplinas.loc[i3,"professor"]))

cargaSemanalUC = []
lUC = len(ucs)
for uc in ucs:
    carga = disciplinas[disciplinas["disciplina"] == uc]["carga_semanal"].iloc[0]
    cargaSemanalUC.append(int(carga))

ucsporProf = {}

for i in range(lUC):
    p = professores.index(disciplinas.loc[i,"professor"])
    if p not in ucsporProf:
        ucsporProf[p] = []
        ucsporProf[p].append(i)
```

```
In [ ]: # Dá map do tipo da sala (por index) ao número da sala
salaNum = {}
counter = 0

for i4 in range(len(salas)):
    if i4 not in salaNum:
        salaNum[i4] = []

        quantidade = salas.loc[i4,"quantidade"]

        for k3 in range(quantidade):
            salaNum[i4].append(counter)
            counter += 1
```

```
In [ ]: tipoSala = {}
for i2 in range(lUC):
    se = disciplinas.loc[i2,"sala_especial"]
    if pd.isna(se):
        tipoSala[i2] = salas.index[salas["tipo"] == "normal"][0]
    else:
        tipoSala[i2] = salas.index[salas["sala"] == se][0]
```

2. Modelação

Índices: turmas $t < T$, disciplinas $u < U$, professores $p < P$, dias $d < D$, horas $h < H$, salas $s < S$.

Parâmetros (vêm dos CSV):

- c_u : carga semanal da disciplina u (`carga_semanal`)
- $dup_u \in \{0, 1\}$: a disciplina u é de duplo período (`duplo_periodo`)
- $Prof_u$: conjunto de professores que podem lecionar u (`profporUcs`)
- Sal_u : conjunto de salas que u pode usar (`salaNum[tipoSala[u]]`)
- $Exc \subseteq P \times D \times H$: tempos em que um professor está indisponível (`exc`)

Variável de decisão:

$x_{t,u,p,d,h,s} = 1 \iff$ a turma t tem a disciplina u com o professor p , no dia d , à hora h , na

O professor p faz parte do índice para permitir **vários professores por disciplina**. A escolha de qual leciona cada turma fica a cargo do solver, condicionada por R0.

Classificação das restrições:

Tipo	Restrições	Efeito
Proibição	R0, R6, R7 (tipo de sala)	fixam variáveis a 0
Limitação	R1, R3, R5, R7 (uma aula por sala)	somas com máximo
Obrigaç�o	R2	igualdade com a carga
Liga�o	R4	liga vari�veis vizinhas entre si

Dimens o do modelo. H  $T \cdot U \cdot P \cdot D \cdot H \cdot S$ vari veis bin rias.   uma formula o densa, simples de escrever e de ler, mas que cresce rapidamente com T , P e S .

```
In [ ]: x = {}

for turma in range(T):
    x[turma] = {}
    for disciplina in range(1UC):
        x[turma][disciplina] = {}
        for professor in range(P):
            x[turma][disciplina][professor] = {}
            for dia in range(D):
                x[turma][disciplina][professor][dia] = {}
                for hora in range(H):
                    x[turma][disciplina][professor][dia][hora] = {}
                    for sala in range(S):
                        x[turma][disciplina][professor][dia][hora][sala] =
horario.IntVar(0,1,'x_%i_%i_%i_%i_%i_%i' %
(turma,disciplina,professor,dia,hora,sala))

def X(t,uc,p,d,h,s):
    # abreviatura
    return x[t][uc][p][d][h][s]
```

Restrição

R0. O professor leciona as disciplinas dele.

$$\forall_{t < T} \cdot \forall_{u < U} \cdot \forall_{p \notin Prof_u} \cdot \sum_{d < D, h < H, s < S} x_{t,u,p,d,h,s} = 0$$

```
In [ ]: for turma_r0 in range(T):  
        for uc_r0 in range(1UC):  
            for prof_r0 in range(P):  
                if prof_r0 not in profporUcs[uc_r0]:  
                    horario.Add(sum([X(turma_r0,uc_r0,prof_r0,dia_r0,hora_r0,s_r0) for  
dia_r0 in range(D) for hora_r0 in range(H) for s_r0 in range(S)]) == 0)
```

R1. Uma turma não pode ter duas aulas em simultâneo.

$$\forall_{t < T} \cdot \forall_{d < D} \cdot \forall_{h < H} \cdot \sum_{u < U, p < P, s < S} x_{t,u,p,d,h,s} \leq 1$$

```
In [ ]: for turma_r1 in range(T):  
        for dia_r1 in range(D):  
            for hora_r1 in range(H):  
                horario.Add(sum([X(turma_r1,uc_r1,prof_r1,dia_r1,hora_r1,s_r1) for  
uc_r1 in range(1UC) for prof_r1 in range(P) for s_r1 in range(S)]) <= 1)
```

R2. Cada disciplina cumpre exatamente a carga semanal definida em disciplinas.csv, para cada turma.

$$\forall_{t < T} \cdot \forall_{u < U} \cdot \sum_{p < P, d < D, h < H, s < S} x_{t,u,p,d,h,s} = c_u$$

```
In [ ]: for turma_r2 in range(T):  
        for uc_r2 in range(1UC):  
            horario.Add(sum([X(turma_r2,uc_r2,prof_r2,d_r2,h_r2,s_r2) for prof_r2 in  
range(P) for d_r2 in range(D) for h_r2 in range(H) for s_r2 in range(S)]) ==  
cargaSemanalUC[uc_r2])
```

R3. No máximo uma aula da mesma disciplina por dia, por turma — exceto disciplinas de duplo período (ver R4), em que o bloco de 2 tempos conta como uma só ocorrência nesse dia.

$$\forall_{t < T} \cdot \forall_{u < U} \cdot \forall_{d < D} \cdot \sum_{p < P, h < H, s < S} x_{t,u,p,d,h,s} \leq 1 + dup_u$$

```
In [ ]: for turma_r3 in range(T):
        for uc_r3 in range(1UC):
            for d_r3 in range(D):
                max_r3 = 1
                if disciplinas.loc[uc_r3,"duplo_perodo"] == "sim":
                    max_r3 = 2
                horario.Add(sum([X(turma_r3,uc_r3,prof_r3,d_r3,h_r3,s_r3) for prof_r3
in range(P) for s_r3 in range(S) for h_r3 in range(H)]) <= max_r3)
```

R4. Disciplinas marcadas duplo_perodo=sim só podem ser dadas em blocos de 2 tempos consecutivos, no mesmo dia (nunca um tempo isolado).

Cada tempo ocupado tem de ter um vizinho imediato (antes ou depois) com o mesmo professor e a mesma sala, com $x_{t,u,p,d,-1,s} = x_{t,u,p,d,H,s} = 0$:

$$\forall_{t < T} \cdot \forall_{u: dup_u=1} \cdot \forall_{p < P} \cdot \forall_{d < D} \cdot \forall_{s < S} \cdot \forall_{h < H} \cdot x_{t,u,p,d,h,s} \leq x_{t,u,p,d,h-1,s} + x_{t,u,p,d,h+1,s}$$

Juntamente com a R3 (no máximo 2 tempos por dia nestas disciplinas), isto obriga a um único bloco de exatamente 2 tempos consecutivos.

```
In [ ]: for turma_r4 in range(T):
        for uc_r4 in range(1UC):
            if disciplinas.loc[uc_r4, "duplo_perodo"] == "sim":
                for prof_r4 in range(P):
                    for d_r4 in range(D):
                        for s_r4 in range(S):
                            for h_r4 in range(H):
                                antes_r4 = X(turma_r4, uc_r4, prof_r4, d_r4, h_r4 - 1,
s_r4) if h_r4 > 0 else 0
                                depois_r4 = X(turma_r4, uc_r4, prof_r4, d_r4, h_r4 +
1, s_r4) if h_r4 < H - 1 else 0
                                horario.Add(X(turma_r4, uc_r4, prof_r4, d_r4, h_r4,
s_r4) <= antes_r4 + depois_r4)
```

R5. Um professor não pode dar duas aulas em simultâneo, mesmo que sejam a turmas ou disciplinas diferentes.

$$\forall_{p < P} \cdot \forall_{d < D} \cdot \forall_{h < H} \cdot \sum_{t < T, u < U, s < S} x_{t,u,p,d,h,s} \leq 1$$

```
In [ ]: for prof_r5 in range(P):
        for d_r5 in range(D):
            for h_r5 in range(H):
                horario.Add(sum([X(t_r5, uc_r5, prof_r5, d_r5, h_r5, s_r5) for t_r5 in
range(T) for uc_r5 in range(1UC) for s_r5 in range(S)]) <= 1)
```

R6. Um professor só pode dar aulas nos tempos em que está disponível (disponibilidade_excecoes.csv).

$$\forall_{(p,d,h) \in Exc} \cdot \sum_{t < T, u < U, s < S} x_{t,u,p,d,h,s} = 0$$

```
In [ ]: for prof_r6, excecoes_prof_r6 in exc.items():
        for dia_exc_r6, hora_exc_r6 in excecoes_prof_r6:
            ucs_do_prof_r6 = ucsporProf[prof_r6]
            for uc_r6 in ucs_do_prof_r6:
                horario.Add(sum([X(t_r6, uc_r6, prof_r6, dia_exc_r6, hora_exc_r6,
s_r6) for t_r6 in range(T) for s_r6 in range(S)]) == 0)
```

R7. Cada aula ocupa uma sala. Disciplinas com sala_especial só podem usar salas desse tipo; as restantes usam salas normal. Em nenhum tempo o número de aulas a decorrer num tipo de sala pode exceder a quantidade desse tipo definida em salas.csv.

1. A disciplina só usa salas do seu tipo (proibição):

$$\forall_{t < T} \cdot \forall_{u < U} \cdot \forall_{p < P} \cdot \forall_{d < D} \cdot \forall_{h < H} \cdot \forall_{s \notin Sal_u} \cdot x_{t,u,p,d,h,s} = 0$$

2. Cada sala tem, no máximo, uma aula em cada tempo (limitação). Isto implica que o número de aulas num tipo de sala nunca excede a quantidade desse tipo:

$$\forall_{d < D} \cdot \forall_{h < H} \cdot \forall_{s < S} \cdot \sum_{t < T, u < U, p < P} x_{t,u,p,d,h,s} \leq 1$$

```
In [ ]: # 1. Garantir que a UC só usa o tipo de sala permitido
for t1_r7 in range(T):
    for uc1_r7 in range(1UC):
        sala_exigida = tipoSala[uc1_r7]
        for prof1_r7 in range(P):
            for d1_r7 in range(D):
                for h1_r7 in range(H):
                    for s1_r7 in range(S):
                        if s1_r7 not in salaNum[sala_exigida]:
                            horario.Add(X(t1_r7, uc1_r7, prof1_r7, d1_r7, h1_r7,
s1_r7 ) == 0)
```

```
# 2. Apenas 1 aula por sala em simultâneo
for d2_r7 in range(D):
    for h2_r7 in range(H):
        for s2_r7 in range(S):
            horario.Add(sum([X(t2_r7, uc2_r7, prof2_r7, d2_r7, h2_r7, s2_r7) for
t2_r7 in range(T) for uc2_r7 in range(1UC) for prof2_r7 in range(P)]) <= 1)
```

O1. Minimizar o número total de "buracos" no horário de cada professor — um buraco é um tempo livre, no meio do dia, entre a primeira e a última aula desse professor nesse dia.

Seja $y_{p,d,h} = \sum_{t,u,s} x_{t,u,p,d,h,s} \in \{0, 1\}$ (o professor p tem aula no tempo h do dia d ; é 0 ou 1 por R5). Para contar os buracos em forma linear, acrescentam-se três famílias de variáveis binárias:

- $pre_{p,d,h}$: há aula em algum tempo $\leq h$
- $pos_{p,d,h}$: há aula em algum tempo $\geq h$
- $ent_{p,d,h}$: o tempo h está entre a primeira e a última aula do dia

$$pre_{p,d,h} \geq y_{p,d,h}, \quad pre_{p,d,h} \geq pre_{p,d,h-1}$$

$$pos_{p,d,h} \geq y_{p,d,h}, \quad pos_{p,d,h} \geq pos_{p,d,h+1}$$

$$ent_{p,d,h} \geq pre_{p,d,h} + pos_{p,d,h} - 1$$

Um buraco é um tempo entre a primeira e a última aula sem aula, logo

$$\min \sum_{p < P, d < D, h < H} (ent_{p,d,h} - y_{p,d,h})$$

Como $\sum y$ é constante (pela R2), é equivalente minimizar $\sum ent$.

```
In [ ]: # y[p,d,h] = 1 se o professor p tem aula no dia d, à hora h (0 ou 1, por R5)
# pre = há aula em algum tempo <= h ; pos = há aula em algum tempo >= h
# ent = o tempo h está entre a primeira e a última aula do dia
ent_o1 = []

for p_o1 in range(P):
    # só as disciplinas deste professor (para as outras, x já é 0 por R0)
    ucs_o1 = [uc_o1 for uc_o1 in range(1UC) if p_o1 in profporUcs[uc_o1]]

    for d_o1 in range(D):
        pre_o1 = [horario.IntVar(0, 1, f"pre_{p_o1}_{d_o1}_{h}") for h in
range(H)]
        pos_o1 = [horario.IntVar(0, 1, f"pos_{p_o1}_{d_o1}_{h}") for h in
range(H)]
        entre_o1 = [horario.IntVar(0, 1, f"ent_{p_o1}_{d_o1}_{h}") for h in
range(H)]
```



```
In [ ]: solucao = obter_solucao(
        horario, X, turmas, ucs, T, LUC, P, D, H, S
    )
```

Solução ótima encontrada!

Representação dos dados

```
In [ ]: def preparar_horario(solucao):
        return (
            pd.DataFrame(solucao)
            .sort_values(["dia", "hora"])
            .reset_index(drop=True)
        )

def obter_turmas(horario_df):
    return sorted(horario_df["turma"].unique())

def obter_professores(horario_df):
    return sorted(horario_df["professor"].unique())

def criar_dropdown(opcoes, label):
    return mo.ui.dropdown(
        options=opcoes,
        value=opcoes[0],
        label=label,
    )

def criar_grelha(df, texto, D, H, dias_nomes, horas_nomes):
    df = df.copy()
    df["celula"] = df.apply(texto, axis=1)

    grelha = df.pivot_table(
        index="hora",
        columns="dia",
        values="celula",
        aggfunc=lambda x: " / ".join(x),
    )

    grelha = (
        grelha
        .reindex(index=range(H), columns=range(D))
        .fillna("")
    )

    grelha.index = horas_nomes
    grelha.columns = dias_nomes
    grelha.index.name = "Hora"

    return grelha.reset_index()
```

```

def criar_grelha_turma(horario_df, turma, D, H, dias_nomes, horas_nomes):
    df = horario_df[horario_df["turma"] == turma]

    return criar_grelha(
        df,
        lambda r: f"{r['disciplina']} ({r['sala']}) - {r['professor']}",
        D,
        H,
        dias_nomes,
        horas_nomes,
    )

def criar_grelha_professor(horario_df, professor, D, H, dias_nomes, horas_nomes):
    df = horario_df[horario_df["professor"] == professor]

    return criar_grelha(
        df,
        lambda r: f"{r['disciplina']} ({r['sala']}) - {r['turma']}",
        D,
        H,
        dias_nomes,
        horas_nomes,
    )

def criar_tabela(grelha_df, H):
    return mo.ui.table(
        grelha_df,
        selection=None,
        page_size=H,
    )

```

Turma: 7ªA ▾

Professor: Prof. Albertina ▾

Q Search...					
Hora str	Segunda str	Terça str	Quarta str	Quinta str	Sexta str
8:00	Matemática (3) - Prof. Ana			História (4) - Prof. Diana	
9:00	História (3) - Prof. Isabela	Português (6) - Prof. Joana	Português (0) - Prof. Joana	Português (1) - Prof. Joana	Matemática (4) - Prof. Ana
10:00		Inglês (6) - Prof. Diana	Matemática (2) - Prof. Ana		Português (0) - Prof. Joana
11:00	Educação Física (11) - Prof. Eduardo	Matemática (3) - Prof. Pedro	Ciências (8) - Prof. Carlota	Inglês (3) - Prof. Diana	Ciências (8) - Prof. Carla
12:00	Educação Física (11) - Prof. Eduardo	Ciências (8) - Prof. Carla			
5 rows, 6 columns			No selection		

Q Search...					
Hora str	Segunda str	Terça str	Quarta str	Quinta object	Sexta str
8:00	Inglês (2) - 7ºB	Inglês (3) - 7ºB			Inglês (0) - 9ºB
9:00		Inglês (0) - 8ºA	Inglês (3) - 8ºB		
10:00					
11:00					
12:00					
5 rows, 6 columns			No selection		

4. Validação automática (R0–R7)

A função `verificar_horario` recebe **apenas** o horário gerado (a lista `solucao`) e os CSV, e recalcula cada restrição diretamente sobre as aulas. Não usa o modelo, o solver nem as estruturas de índices construídas para ele (`x`, `profporUcs`, `ucsporProf`, `tipoSala`). Assim, um erro de modelação, como um índice trocado numa restrição, não é escondido por um erro igual na verificação: o solver gera um horário que obedece à restrição errada e o verificador, que a calcula de outra forma, assinala-o. Devolve a lista de violações (`vazia` = horário válido).

Restrição	O que se verifica sobre as aulas
R0	o professor de cada aula leciona essa disciplina (segundo <code>disciplinas.csv</code>)
R1	não há duas aulas com o mesmo (turma, dia, hora)
R2	para cada (turma, disciplina), o número de aulas é igual à carga semanal
R3	por (turma, disciplina, dia), no máximo 1 aula (2 se duplo período)
R4	em duplo período, as aulas do dia são exatamente 2, em horas consecutivas, com o mesmo professor e sala
R5	não há duas aulas com o mesmo (professor, dia, hora)
R6	nenhuma aula cai num (professor, dia, hora) de <code>disponibilidade_excecoes.csv</code>
R7	a sala é do tipo exigido pela disciplina e não há duas aulas com o mesmo (sala, dia, hora)

Tal como o modelo, o R2 assume que todas as turmas têm todas as disciplinas.

Teste negativo. Um verificador que devolve sempre "tudo certo" também passaria o teste anterior. Por isso, `executar_testes_negativos` pega no horário gerado e estraga-o de propósito, uma restrição de cada vez (por exemplo, move uma aula para um tempo de exceção do professor), e confirma que o verificador deteta essa violação. Isto cobre pelo menos um caso de cada restrição.

Utilização de IA. Para esta secção pedimos ao Gemini que escrevesse a verificação automática; a conversa está disponível [aqui](#). Limitámo-nos a inspirar-nos na solução proposta e alterámo-la de acordo com o que pretendíamos para este trabalho.

```
In [ ]: def construir_contexto(disciplinas, excecoes, salas):
    """Reconstrói, só a partir dos CSV, tudo o que é preciso para verificar um
    horário.
    Não usa nenhuma estrutura do modelo (x, profporUcs, ucsporProf, tipoSala,
    ...)."""
    dias = {"Seg": 0, "Ter": 1, "Qua": 2, "Qui": 3, "Sex": 4}

    # uma linha por disciplina (a primeira), como no resto do notebook
    primeira = disciplinas.drop_duplicates("disciplina").set_index("disciplina")

    # salas: a sala global s pertence à linha de salas.csv cujo intervalo a contém
    intervalos, n = [], 0
    for q in salas["quantidade"]:
        intervalos.append(set(range(n, n + int(q))))
        n += int(q)

    linha_normal = salas.index[salas["tipo"] == "normal"][0]
    sala_exigida = {}
    for disc, se in primeira["sala_especial"].items():
        linha = linha_normal if pd.isna(se) else salas.index[salas["sala"] == se]
        sala_exigida[disc] = intervalos[linha]

    return {
        "profs_da_disc": disciplinas.groupby("disciplina")
        ["professor"].apply(set).to_dict(),
        "carga": primeira["carga_semanal"].astype(int).to_dict(),
        "duplo": (primeira["duplo_periodo"] == "sim").to_dict(),
        "excecoes": {(r.professor, dias[r.dia], int(r.periodo) - 1) for r in
        excecoes.itertuples()},
        "sala_exigida": sala_exigida,
        "n_salas": n,
    }

def verificar_horario(solucao, disciplinas, turmas, excecoes, salas):
    """Devolve a lista de violações de R0-R7 (lista vazia = horário válido).
    Cada aula é um dict com: turma, disciplina, professor, dia, hora, sala."""
    from collections import Counter, defaultdict

    ctx = construir_contexto(disciplinas, excecoes, salas)
    viol = []
```

```

# R0: o professor leciona a disciplina
for a in solucao:
    if a["professor"] not in ctx["profs_da_disc"].get(a["disciplina"], set()):
        viol.append(f"R0: {a['professor']} não leciona {a['disciplina']}")

# R1: uma turma não tem duas aulas em simultâneo
for (t, d, h), n in Counter((a["turma"], a["dia"], a["hora"])) for a in solucao).items():
    if n > 1:
        viol.append(f"R1: turma {t} tem {n} aulas no dia {d}, hora {h}")

# R2: cada disciplina cumpre a carga semanal, para cada turma
contagem = Counter((a["turma"], a["disciplina"]))
for t in turmas["turma"]:
    for disc, carga in ctx["carga"].items():
        if contagem[(t, disc)] != carga:
            viol.append(f"R2: turma {t}, {disc}: {contagem[(t, disc)]} aulas (carga {carga})")

# R3 e R4: agrupar as aulas por (turma, disciplina, dia)
por_dia = defaultdict(list)
for a in solucao:
    por_dia[(a["turma"], a["disciplina"], a["dia"])].append(a)

for (t, disc, d), aulas in por_dia.items():
    duplo = ctx["duplo"].get(disc, False)

    # R3: no máximo 1 aula por dia (2 se duplo período)
    if len(aulas) > (2 if duplo else 1):
        viol.append(f"R3: turma {t}, {disc}, dia {d}: {len(aulas)} aulas")

    # R4: duplo período = bloco de 2 tempos consecutivos, mesmo professor e mesma sala
    if duplo:
        horas = sorted(a["hora"] for a in aulas)
        mesmo_prof = len({a["professor"] for a in aulas}) == 1
        mesma_sala = len({a["sala"] for a in aulas}) == 1
        if not (len(aulas) == 2 and horas[1] - horas[0] == 1 and mesmo_prof and mesma_sala):
            viol.append(f"R4: turma {t}, {disc}, dia {d}: não é um bloco válido de 2 tempos (horas {horas})")

# R5: um professor não dá duas aulas em simultâneo
for (p, d, h), n in Counter((a["professor"], a["dia"], a["hora"])) for a in solucao).items():
    if n > 1:
        viol.append(f"R5: {p} tem {n} aulas no dia {d}, hora {h}")

# R6: o professor só dá aulas quando está disponível
for a in solucao:
    if (a["professor"], a["dia"], a["hora"]) in ctx["excecoes"]:
        viol.append(f"R6: {a['professor']} está indisponível no dia {a['dia']}, hora {a['hora']}")

# R7: a sala é do tipo exigido e cada sala tem no máximo uma aula por tempo
for a in solucao:

```

```

        if a["sala"] not in ctx["sala_exigida"].get(a["disciplina"], set()):
            viol.append(f"R7: {a['disciplina']} (turma {a['turma']}) está na sala {a['sala']}, de tipo errado")
        for (s, d, h), n in Counter((a["sala"], a["dia"], a["hora"])) for a in solucao).items():
            if n > 1:
                viol.append(f"R7: sala {s} tem {n} aulas no dia {d}, hora {h}")

    return viol

def executar_testes_negativos(solucao, disciplinas, turmas, excecoes, salas):
    """Estraga de propósito um horário válido, uma restrição de cada vez, e confirma que o verificador detecta essa violação. Devolve uma tabela com o resultado de cada teste."""
    ctx = construir_contexto(disciplinas, excecoes, salas)
    todos_profs = set(disciplinas["professor"])
    resultados = []

    def testar(codigo, descricao, aulas):
        if aulas is None:
            resultados.append({"Restrição": codigo, "Corrupção aplicada": descricao, "Detetada": "n/a (sem caso nos dados)"})
            return
        viol = verificar_horario(aulas, disciplinas, turmas, excecoes, salas)
        resultados.append({"Restrição": codigo, "Corrupção aplicada": descricao, "Detetada": any(v.startswith(codigo + ":") for v in viol)})

    def copia():
        return [dict(a) for a in solucao]

    def primeira(cond):
        return next((i for i, a in enumerate(solucao) if cond(a)), None)

    # R0: trocar o professor de uma aula por outro que não leciona a disciplina
    i = primeira(lambda a: todos_profs - ctx["profs_da_disc"][a["disciplina"]])
    if i is None:
        testar("R0", "professor que não leciona a disciplina", None)
    else:
        aulas = copia()
        aulas[i]["professor"] = sorted(todos_profs - ctx["profs_da_disc"][aulas[i]["disciplina"]])[0]
        testar("R0", "professor que não leciona a disciplina", aulas)

    # R1: duplicar uma aula (a turma fica com duas aulas no mesmo tempo)
    aulas = copia()
    aulas.append(dict(aulas[0]))
    testar("R1", "aula duplicada no mesmo tempo da mesma turma", aulas)

    # R2: remover uma aula (a carga deixa de se cumprir)
    testar("R2", "aula removida", copia()[1:])

    # R3: duas aulas no mesmo dia de uma disciplina sem duplo período
    i = primeira(lambda a: not ctx["duplo"][a["disciplina"]])

```

```

if i is None:
    testar("R3", "2 aulas no mesmo dia, sem duplo período", None)
else:
    aulas = copia()
    extra = dict(aulas[i])
    extra["hora"] = 0 if extra["hora"] != 0 else 1
    aulas.append(extra)
    testar("R3", "2 aulas no mesmo dia, sem duplo período", aulas)

# R4: partir um bloco de duplo período (fica um tempo isolado)
i = primeira(lambda a: ctx["duplo"][a["disciplina"]])
if i is None:
    testar("R4", "bloco de duplo período partido", None)
else:
    aulas = copia()
    del aulas[i]
    testar("R4", "bloco de duplo período partido", aulas)

# R5: o mesmo professor numa segunda turma, ao mesmo tempo
aulas = copia()
extra = dict(aulas[0])
extra["turma"] = "turma_fantasma"
aulas.append(extra)
testar("R5", "professor com duas aulas ao mesmo tempo", aulas)

# R6: mover uma aula para um tempo de exceção do professor
alvo = next(((p, d, h, k) for (p, d, h) in sorted(ctx["excecoes"])
            for k, a in enumerate(solucao) if a["professor"] == p), None)
if alvo is None:
    testar("R6", "aula num tempo de exceção do professor", None)
else:
    p, d, h, k = alvo
    aulas = copia()
    aulas[k]["dia"], aulas[k]["hora"] = d, h
    testar("R6", "aula num tempo de exceção do professor", aulas)

# R7: pôr uma aula numa sala de tipo errado
i = primeira(lambda a: len(ctx["sala_exigida"][a["disciplina"]]) <
ctx["n_salas"])
if i is None:
    testar("R7", "sala de tipo errado", None)
else:
    aulas = copia()
    errada = next(s for s in range(ctx["n_salas"]) if s not in
ctx["sala_exigida"][aulas[i]["disciplina"]])
    aulas[i]["sala"] = errada
    testar("R7", "sala de tipo errado", aulas)

return pd.DataFrame(resultados)

```

```

In [ ]: # Verificação do horário gerado
if not solucao:
    _out = mo.md("Sem solução para verificar.")
else:
    _viol = verificar_horario(solucao, disciplinas, turmas, excecoes, salas)
    if not _viol:

```

```

_out = mo.md(f"O horário cumpre R0-R7 ({len(solucao)} aulas
verificadas).")
else:
    _out = mo.vstack([
        mo.md(f"{len(_viol)} violações encontradas:"),
        mo.md("\n".join(f"- {v}" for v in _viol[:50])),
    ])
_out

```

O horário cumpre R0–R7 (102 aulas verificadas).

```

In [ ]: # Testes negativos: cada corrupção tem de ser detetada
mo.stop(not solucao, mo.md("Sem solução para estragar."))
testes_negativos = executar_testes_negativos(solucao, disciplinas, turmas,
excecoes, salas)
testes_negativos

```

Q Search...

Restrição str	Corrupção aplicada str	Detetada bool	
R0	professor que não leciona a disciplina	True	
R1	aula duplicada no mesmo tempo da mesma turma	True	
R2	aula removida	True	
R3	2 aulas no mesmo dia, sem duplo período	True	
R4	bloco de duplo período partido	True	
R5	professor com duas aulas ao mesmo tempo	True	
R6	aula num tempo de exceção do professor	True	
R7	sala de tipo errado	True	

8 rows, 3 columns No selection

5. Âmbito e limitações

O que o notebook faz. Lê os dados, constrói o modelo com as restrições R0–R7, minimiza os buracos dos professores (O1), resolve com o SCIP, apresenta o horário por turma e por professor e verifica automaticamente que o horário gerado cumpre R0–R7 (com testes negativos).

Construção incremental (R9): não implementada, por escolha. Optámos por não a fazer devido à complexidade que levanta. Exigiria, no mínimo, definir e medir o que é uma "aula alterada" entre dois horários, acrescentar ao objetivo um termo de estabilidade face ao horário anterior (com um peso a calibrar contra os buracos) e comparar rigorosamente o tempo com o de resolver de raiz. Preferimos concentrar o trabalho num modelo base correto e bem documentado a entregar uma solução incremental incompleta.

Outras limitações.

- **Formulação densa.** O número de variáveis cresce com o produto de todos os índices. Criar variáveis apenas para combinações permitidas ($p \in Prof_u$, $s \in Sal_u$) reduziria muito o modelo.
- **Solver genérico.** O SCIP serve bem para este tamanho, mas o CP-SAT poderá escalar melhor.